

অ্যান্ড্রয়েড মোবাইল-চালিত স্বায়ত্তশাসিত অ্যাপ্লিকেশন ইঞ্জিন (L9_META_DESIGNER) নির্মাণের ইউনিভার্সাল রূপান্তর ও গাইডবুক

কম্পিউটার বা ল্যাপটপের কোনো প্রকার ব্যবহার ছাড়াই শুধুমাত্র একটি অ্যান্ড্রয়েড মোবাইল ডিভাইস ব্যবহার করে সম্পূর্ণ স্বয়ংক্রিয়ভাবে ওয়েব অ্যাপ্লিকেশনের জিপ (ZIP) ফাইল বা রূপান্তর থেকে সরাসরি ইনস্টলযোগ্য অ্যান্ড্রয়েড এপিকে (APK) তৈরি করার প্রক্রিয়াটি আধুনিক ক্লাউড কম্পিউটিং এবং এজেন্টিক এআই (Agentic AI) প্রযুক্তির এক অনন্য সমন্বয়। L9_META_DESIGNER প্রজেক্টের অধীনে প্রস্তাবিত এই অ্যাপ্লিকেশন ইঞ্জিনটি মূলত ফায়ারবেস স্টুডিও অ্যাপ্লিকেশন বিল্ডার (Firebase Studio Application Builder) বা এআই স্টুডিও অ্যাপ বিল্ডার (AI Studio App Builder) হিসেবে কাজ করবে। এই সিস্টেমের মূল চালিকাশক্তি হলো গুগল জুলস (Google Jules) নামক স্বায়ত্তশাসিত ক্লাউড কোডিং এজেন্ট এবং ক্যাপাসিটর (Capacitor) ভিত্তিক সিআই/সিডি (CI/CD) অটোমেশন পাইপলাইন। এই সিস্টেমের সম্পূর্ণ কার্যপ্রণালীটি একটি মোবাইল-অনলি ডেভেলপমেন্ট প্যারাডাইম প্রতিষ্ঠা করে, যেখানে ক্লাউড কম্পিউটেশনাল শক্তি ব্যবহার করে মোবাইলের হার্ডওয়্যার সীমাবদ্ধতা অতিক্রম করা সম্ভব হয়। নিচে এই ইঞ্জিনের সামগ্রিক উপাদান এবং তাদের পারস্পরিক সম্পর্ক বিস্তারিতভাবে উপস্থাপন করা হলো।

সিস্টেম আর্কিটেকচার এবং ইউনিভার্সাল রূপান্তর

মোবাইল-চালিত এআই অ্যাপ্লিকেশন ইঞ্জিনের রূপান্তরটি মূলত পাঁচটি প্রধান স্তরের সমন্বয়ে গঠিত। প্রথম স্তরে ব্যবহারকারী তার অ্যান্ড্রয়েড মোবাইলের ব্রাউজার থেকে একটি প্রগ্রেসিভ ওয়েব অ্যাপ (PWA) কনসোলে অ্যাক্সেস করেন, যা [ফায়ারবেস হোস্টিং \(Firebase Hosting\)](#) দ্বারা হোস্ট করা হয়। দ্বিতীয় স্তরে, আপলোড করা জিপ ফাইল বা টেমপ্লেট রূপান্তরটি ফায়ারবেস ক্লাউড স্টোরেজে সংরক্ষিত হয় এবং ফায়ারবেস রিয়েলটাইম ডাটাবেস বা ফায়ারস্টোরের মাধ্যমে মেটাডেটা প্রসেস করা হয়। তৃতীয় স্তরে, এআই কোডিং এজেন্ট গুগল জুলস গিটহাব রিপোজিটরির সাথে সংযুক্ত হয়ে সোর্স কোড জেনারেট বা মডিফাই করে। চতুর্থ স্তরে, গিটহাব অ্যাকশনস রানার ক্যাপাসিটর ফ্রেমওয়ার্ক ব্যবহার করে এই কোডবেসকে অ্যান্ড্রয়েড নেটিভ প্রজেক্টে রূপান্তর করে এবং গ্র্যাডল (Gradle) বিল্ডার মাধ্যমে রিলিজ এপিকে কম্পাইল করে। পঞ্চম ও শেষ স্তরে, ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশন ব্যবহারের মাধ্যমে সরাসরি ব্যবহারকারীর মোবাইলে অ্যাপটি ডাউনলোডের জন্য ওটিএ (Over-The-Air) পুশ করা হয়। নিচের সারণীতে এই আর্কিটেকচারের প্রধান টেকনিক্যাল স্পেসিফিকেশন এবং রিসোর্স কনফিগারেশন সংক্ষেপে তুলে ধরা হলো:

উপাদান (Component)	টেকনোলজি স্ট্যাক (Tech Stack)	ভূমিকা ও উদ্দেশ্য (Role & Purpose)	তথ্যসূত্র ও গাইড লিঙ্ক
মোবাইল কনসোল (Frontend)	React / Vue PWA	মোবাইল স্ক্রিনে জিপ ফাইল আপলোড এবং রূপান্তর ইনপুট নেওয়া	https://firebase.google.com/docs/web/setup
ডাটা ও ক্লাউড স্টোরেজ	Cloud Firestore / Storage	জিপ ফাইল এবং আর্কিটেকচারাল ম্যাপিং ডেটা ক্যাশ ও স্টোর করা	https://firebase.google.com/docs/reference/js
এআই কোডিং ইঞ্জিন	Google Jules REST API	কোডবেস রিডিং, নতুন এপিআই যুক্ত করা এবং টেস্ট জেনারেশন	https://jules.google/docs/api/reference/overview/
নেটিভ অ্যান্ড্রয়েড বাইন্ডিং	Capacitor Framework	জাভাস্ক্রিপ্ট/ওয়েব প্রজেক্টকে অ্যান্ড্রয়েড কোডবেসে রূপান্তর	Understand Firebase Android
সিআই/সিডি অটোমেশন	GitHub Actions Workflow	গ্র্যাডল দিয়ে রিলিজ এপিকে কম্পাইল ও সাইন করা	https://firebase.google.com/docs/android/setu

উপাদান (Component)	টেকনোলজি স্ট্যাক (Tech Stack)	ভূমিকা ও উদ্দেশ্য (Role & Purpose)	তথ্যসূত্র ও গাইড লিঙ্ক
মোবাইল রিলিজ ডিস্ট্রিবিউশন	Firebase App Distribution	এক ক্লিকে সরাসরি মোবাইলে অ্যান্ড্রয়েড এপিকে ডাউনলোড লিংক পাঠানো	p) (https://github.com/marketplace/actions/firebase-app-distribution))

ধাপ-ভিত্তিক ক্লিক-বাই-ক্লিক বাস্তবায়ন গাইডবুক

এই স্বায়ত্তশাসিত ইঞ্জিনটি অ্যান্ড্রয়েড মোবাইল দিয়ে সেটআপ এবং পরিচালনা করার জন্য প্রতিটি ধাপ অত্যন্ত সূক্ষ্মভাবে ডিজাইন করা হয়েছে। নিচে সম্পূর্ণ সেটআপ গাইড ক্লিক-বাই-ক্লিক ক্লে আকারে উপস্থাপন করা হলো।

ধাপ ১: ফায়ারবেস ক্লাউড ইনফ্রাস্ট্রাকচার এবং হোস্টিং সেটআপ

- অ্যান্ড্রয়েড টার্মিনাল কনফিগারেশন: মোবাইলে গুগল প্লে স্টোর থেকে **Termux** অ্যাপ্লিকেশনটি ইনস্টল করতে হবে। টার্মিনালটি ওপেন করে নোড এনভায়রনমেন্ট কনফিগার করার জন্য নিচের কমান্ডগুলো রান করতে হবে:

```
pkg update && pkg upgrade
pkg install nodejs
```
- ফায়ারবেস সিআইআই অ্যাক্টিভেশন: টার্মিনালে ফায়ারবেস টুলস গ্লোবালি ইনস্টল করতে হবে:

```
npm install -g firebase-tools
```
- অথেন্টিকেশন প্রোটোকল: মোবাইলে ফায়ারবেস ক্লাউডে লগইন করার জন্য রান করতে হবে:

```
firebase login --no-localhost
```

টার্মিনালে আসা অথেন্টিকেশন লিঙ্কটি কপি করে মোবাইল ব্রাউজারে (যেমন: Chrome) পেস্ট করে গুগল অ্যাকাউন্টের মাধ্যমে পারমিশন কনফার্ম করতে হবে। ব্রাউজার থেকে প্রাপ্ত ওথ (OAuth) টোকেনটি টার্মিনালে ইনপুট দিয়ে সফলভাবে লগইন সম্পন্ন করতে হবে।
- প্রজেক্ট ইনিশিয়ালাইজেশন: ব্রাউজারে [ফায়ারবেস কনসোল](#) ওপেন করে একটি নতুন প্রজেক্ট তৈরি করতে হবে। এরপর Termux টার্মিনালে প্রজেক্ট ডিরেক্টরিতে গিয়ে রান করতে হবে:

```
firebase init hosting
```

এখানে প্রজেক্টটি সিলেক্ট করতে হবে এবং ফ্রন্টএন্ড কোডের পাবলিক ডিরেক্টরি সেট করতে হবে। ব্রাউজারের সাথে সামঞ্জস্যতা বজায় রাখার জন্য ফায়ারবেস মডিউলার এসডিকে ব্যবহার করা বাঞ্ছনীয়, যার বিস্তারিত গাইডলাইন(<https://firebase.google.com/docs/web/modular-upgrade>) থেকে অনুসরণ করা যাবে। মোবাইল ব্রাউজারের কর্মক্ষমতা বাড়াতে(<https://firebase.google.com/docs/web/best-practices>) এবং সিডিএন ভিত্তিক(<https://firebase.google.com/docs/web/alt-setup>) ব্যবহার করা উচিত। অ্যাপ্লিকেশন বিল্ডার কনসোলটি হোস্ট করার জন্য রান করতে হবে:

```
firebase deploy --only hosting
```

ধাপ ২: গুগল জুলস এপিআই ইন্টিগ্রেশন এবং স্বায়ত্তশাসিত সেশন ট্রিগার

- এপিআই কী জেনারেশন: মোবাইল ব্রাউজার ব্যবহার করে jules.google.com পোর্টালে প্রবেশ করতে হবে এবং গুগল অ্যাকাউন্টের মাধ্যমে লগইন করতে হবে। **Settings** মেনুতে গিয়ে **Generate API Key** বাটনে ক্লিক করতে হবে এবং জেনারেট হওয়া ইউনিক কী-টি নিরাপদ স্থানে সেভ করে রাখতে হবে।
- গিটহাব সোর্স কানেকশন: জুলস ড্যাশবোর্ড থেকে **Connect to GitHub account** এ ক্লিক করে প্রয়োজনীয়

পারমিশন সহ গিটহাব রিপোজিটরির অ্যাক্সেস দিতে হবে।

- সেশন ডিগার স্ক্রিপ্ট: ফায়ারবেস অ্যাপ বিল্ডার ইঞ্জিন থেকে জুলসের এআই কোডিং প্রসেস সক্রিয় করার জন্য নিম্নোক্ত এইচটিটিপি পোস্ট (HTTP POST) রিকোয়েস্টটি মোবাইলের ব্যাকএন্ড রানার বা ক্লাউড ফাংশন থেকে পাঠানো হবে :
 - রিকোয়েস্ট ইউআরএল: <https://jules.googleapis.com/v1alpha/sessions> * হেডার কনফিগারেশন:
 - Content-Type: application/json
 - X-Goog-API-Key: YOUR_JULES_API_KEY (ধাপ ২ থেকে প্রাপ্ত কী)
 - পেলোড বডি (JSON):

```
{  "prompt": "Create an elegant mobile dashboard application using HTML5, CSS3, and modern Javascript, then optimize it for Capacitor runtime integration.",  "sourceContext": {    "source": "sources/github/your-github-username/your-target-repo",    "githubRepoContext": {      "startingBranch": "main"    }  },  "automationMode": "AUTO_CREATE_PR",  "title": "Automated Web App Generation"}
```

এই এপিআই রিকোয়েস্টের ফলে গুগল জুলস স্বয়ংক্রিয়ভাবে ক্লাউড VM স্পিন-আপ করবে, সম্পূর্ণ প্রজেক্টের কোড অ্যানালাইসিস করবে এবং ব্যবহারকারীর গিটহাব ব্রাঞ্চে রিয়েল-টাইম কোড পরিবর্তন করে একটি পুল রিকোয়েস্ট (PR) ওপেন করবে।

ধাপ ৩: গিটহাব অ্যাকশনস সিআই/সিডি এবং কোড সাইনিং কনফিগারেশন

- রিলিজ কি-স্টোর জেনারেশন: Termux টার্মিনালে অ্যান্ড্রয়েড রিলিজ সাইনিং কী (Keystore) তৈরি করতে হবে :

```
keytool -genkey -v -keystore release.jks -alias release-alias -keyalg RSA -keysize 2048 -validity 10000
```
- বেস-৬৪ রূপান্তর: কি-স্টোর বাইনারি ফাইলটিকে গিটহাবে টেক্সট সিক্রেট হিসেবে স্টোর করার জন্য বেস-৬৪ ফরম্যাটে রূপান্তর করতে হবে :

```
base64 -i release.jks > base64-keystore.txt
```
- গিটহাব সিক্রেট এনভায়রনমেন্ট প্রোভিশনিং: মোবাইল ব্রাউজার দিয়ে গিটহাব প্রজেক্টের **Settings -> Secrets and variables -> Actions** পেজে যেতে হবে এবং **New repository secret** বাটনে ক্লিক করে নিম্নোক্ত ডেটা ইনপুট দিতে হবে :

সিক্রেট ভেরিয়েবলের নাম	সিক্রেটের মান / কনটেন্ট	সিকিউরিটি প্রোটোকল
KEYSTORE_BASE64	base64-keystore.txt থেকে প্রাপ্ত সম্পূর্ণ টেক্সট স্ট্রিং	মেমোরি-অনলি JIT ডিক্রিপশন
KEYSTORE_PASSWORD	কি-স্টোর তৈরির সময় ব্যবহৃত প্রধান সিকিউর পাসওয়ার্ড	রিয়েল-টাইম লগ মাস্কিং
KEY_ALIAS	কি-স্টোরের জন্য ব্যবহৃত এলিয়াস নাম	মেমোরি-অনলি JIT ডিক্রিপশন

সিক্রেট ভেরিয়েবলের নাম	সিক্রেটের মান / কনটেন্ট	সিকিউরিটি প্রোটোকল
	(যেমন: release-alias)	
KEY_PASSWORD	নির্দিষ্ট কি-এলিয়াসের জন্য নির্ধারিত অ্যাক্সেস পাসওয়ার্ড	রিমোল-টাইম লগ মাস্কিং
F[span_31](start_span)[span_31](end_span)IREBASE_APP_ID	ফায়ারবেস অ্যান্ড্রয়েড অ্যাপ সেটিংস থেকে প্রাপ্ত ইউনিক অ্যাপ আইডি	এনক্রিপ্টেড সিক্রেট স্টোরেজ
FIREBASE_SERVICE_ACCOUNT_KEY	জেনারেট করা ক্লাউড সার্ভিস অ্যাকাউন্টের সম্পূর্ণ JSON ফাইল কনটেন্ট	জাস্ট-ইন-টাইম (JIT) আইসোলেশন

- স্বয়ংক্রিয় ওয়ার্কফ্লো স্ক্রিপ্ট: এই পর্যায়ে .github/workflows/capacitor-build.yml ফাইলটি রিপোজিটরিতে যুক্ত করতে হবে যা কোড পুশ হওয়ার সাথে সাথে সোর্স কোডকে ক্যাপাসিটর অ্যান্ড্রয়েড বিল্ড এনভায়রনমেন্টে সিনক্রোনাইজ করবে:

```
name: Autonomous Mobile APK Builder
```

```
on:
```

```
  push:
    branches:
      - main
  repository_dispatch:
    types: [trigger-build]
```

```
jobs:
```

```
  build_and_deploy[span_15] (start_span) [span_15] (end_span) [span_17] (start_span) [span_17] (end_span) oy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- name: Checkout Code
 uses: actions/checkout@v4
- name: Setup Node.js
 uses: actions/setup-node@v4
 with:
 node-version: '20'
- name: Setup Java JDK 17
 uses: actions/setup-java@v4
 with:
 distribution: 'zulu'
 java-version: '17'
- name: Install Node Dependencies
 run: npm ci
- name: Compile Web Assets
 run: npm run build

```

- name: Initialize Capacitor Android Platform
  run: |
    npm install @capacitor/core @capacitor/cli
@capacitor/android
    npx cap sync android

- name: Cache Gradle Dependencies
  uses: actions/cache@v4
  with:
    path: |
      ~/.gradle/caches
      ~/.gradle/wrapper
    key: ${ runner.os }-gradle-${
hashFiles('**/*.gradle*', '**/gradle-wrapper.properties') }

- name: Make Gradle Executable
  run: chmod +x android/gradlew

- name: Decrypt and Import Android Keystore
  run: echo "${ secrets.KEYSTORE_BASE64 }}" | base64 -d >
android/app/release.jks

- name: Build Signed APK
  run: |
    cd android
    ./gradlew assembleRelease
  env:
    RELEASE_STORE_FILE: release.jks
    RELEASE_STORE_PASSWORD: ${ secrets.KEYSTORE_PASSWORD }
    RELEASE_KEY_ALIAS: ${ secrets.KEY_ALIAS }
    RELEASE_KEY_PASSWORD: ${ secrets.KEY_PASSWORD }

- name: Push APK to Firebase App Distribution
  uses: wzieba/Firebase-Distribution-Github-Action@v1
  with:
    appId: ${ secrets.FIREBASE_APP_ID }
    serviceCredentialsFileContent: ${
secrets.FIREBASE_SERVICE_ACCOUNT_KEY }
    file:
android/app/build/outputs/apk/release/app-release.apk
    groups: testers

```

ধাপ ৪: ওটিএ ওয়ান-ক্লিক মোবাইল ইনস্টলেশন

- গিটহাব অ্যাকশনস রানার সফলভাবে রিলিজ এপিকে কম্পাইল করার পর এটি সরাসরি ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশন প্ল্যাটফর্মে আপলোড করে দেবে।
- ব্যবহারকারী বা টেস্টারের অ্যাপ্লয়েড মোবাইলে একটি স্বয়ংক্রিয় ইমেইল নোটিফিকেশন অথবা ফায়ারবেস অ্যাপ

ডিস্ট্রিবিউশন অ্যাপের মাধ্যমে একটি নোটিফিকেশন চলে যাবে।

- ব্যবহারকারী তার অ্যান্ড্রয়েড স্ক্রিনে নোটিফিকেশনের ওপর ক্লিক করে সরাসরি ব্রাউজারের মাধ্যমে কোনো প্রকার থার্ড-পার্টি ক্যাবল সংযোগ বা কম্পিউটার ছাড়াই এপিকে ফাইলটি ওয়ান-ক্লিকে ইনস্টল করে নিতে পারবেন।

প্রযুক্তিগত গ্যাপ বিশ্লেষণ ও এআই অপ্টিমাইজেশন

প্রস্তাবিত মোবাইল-অনলি ডেভেলপমেন্ট ও কোড জেনারেশন আর্কিটেকচারটি অত্যন্ত কার্যকর হলেও, এটি বাস্তবায়নের ক্ষেত্রে কিছু প্রযুক্তিগত চ্যালেঞ্জ বা গ্যাপ বিদ্যমান যা দূর করা অত্যন্ত জরুরি।

গ্যাপ ১: এআই দ্বারা জেনারেট করা কোডের এপিআই লিগ্যাসি এরর

গুগল জুলস বা যেকোনো লার্জ ল্যাঙ্গুয়েজ মডেল (LLM) যখন কোড তৈরি করে, তখন তাদের ট্রেনিং ডেটার সীমাবদ্ধতার কারণে তারা অনেক সময় অবচলিত বা লিগ্যাসি ফায়ারবেস নেমস্পেস এপিআই (Firebase Namespaced APIs) ব্যবহার করে কোড লিখে থাকে, যার ফলে আধুনিক মডিউলার প্রজেক্টে বিল্ড এরর দেখা দিতে পারে।

সমাধান এবং এআই সিআই সেলফ-হিলিং প্রোটোকল: এই প্রযুক্তিগত গ্যাপটি সমাধান করার জন্য গুগল-এর অফিসিয়াল ডেভেলপার নলেজ এপিআই (**Developer Knowledge API**) এবং এমসিপি সার্ভার (**MCP Server**) ব্যবহার করা উচিত। এই টুলটি গুগলের অফিসিয়াল ডেভেলপার ডকুমেন্টেশনের একটি মেশিন-রিডেবল গেটওয়ে হিসেবে কাজ করে যা প্রতি ২৪ ঘন্টায় আপডেট করা হয়। গুগল জুলস এই এপিআই-এর সাথে এমসিপি প্রোটোকলের মাধ্যমে সংযুক্ত হয়ে রিয়েল-টাইমে সবচেয়ে আপডেট হওয়া মডিউলার জাভাস্ক্রিপ্ট কোড, প্রগ্রেসিভ ওয়েব অ্যাপ (PWA) স্ট্রাকচার, কিংবা অ্যান্ড্রয়েড কটলিন এক্সটেনশন (KTX) ফ্রেমওয়ার্ক ব্যবহার করে কোড জেনারেট করবে। এছাড়া এর

মাধ্যমে(<https://firebase.google.com/docs/reference/js>)

এবং(<https://firebase.google.com/docs/reference/android/packages>) সরাসরি কোয়েরি করা সম্ভব, যা এআই কোড জেনারেশনে হ্যালুসিনেশন বা ভুল এপিআই ব্যবহারের সম্ভাবনা সম্পূর্ণ দূর করে।

গ্যাপ ২: গ্র্যাডল কমপ্লায়েন্স এবং গুগল প্লে সার্ভিসেস ডিপেন্ডেন্সি এরর

মোবাইল থেকে সরাসরি এপিকে রান করার সময় অনেক ক্ষেত্রে দেখা যায় যে, তৈরি করা অ্যাপ্লিকেশনের ফায়ারবেস ক্লাউড মেসেজিং বা পুশ নোটিফিকেশন কাজ করছে না। এর মূল কারণ হলো রানটাইমে ডিভাইসে সঠিক প্লে সার্ভিসেস ডিপেন্ডেন্সি অনুপস্থিত থাকা।

সমাধান: গিটহাব অ্যাকশনস কম্পাইলেশন ফেজে নিশ্চিত করতে হবে যে গ্র্যাডল বিল্ড

স্ক্রিপ্টটি([<https://firebase.google.com/docs/android/android-play-services>])(<https://firebase.google.com/docs/android/android-play-services>)) গাইডলাইন কঠোরভাবে অনুসরণ করে বিল্ড কনফিগার করছে।

ত্রুটি এড়াতে কাস্টম সিআই রুল যুক্ত করতে হবে যা বিল্ড চালুর পূর্বে মেটাডেটা স্ক্যান করে সঠিক এসডিকে লাইব্রেরি এবং প্লে সার্ভিস লাইব্রেরির সংস্করণগুলোর সামঞ্জস্যতা নিশ্চিত করবে। এছাড়াও বিল্ড টাইম ট্রাবলশুটিং সহজ

করতে(<https://firebase.google.com/docs/android/troubleshooting-faq>) ডাটাবেসটি এআই এজেন্টের নিজস্ব নলেজবেইসে যুক্ত করে দিতে হবে যাতে এআই নিজে থেকেই গ্র্যাডল কনফ্লিক্টগুলো সমাধান করতে পারে।

গ্যাপ ৩: ক্রস-প্ল্যাটফর্ম হাইব্রিড জেনারেশন লিমিটেশন

শুধুমাত্র ক্যাপাসিটর এবং এইচটিএমএল/জাভাস্ক্রিপ্ট দিয়ে এন্টারপ্রাইজ গ্রেড জটিল নেটিভ অ্যান্ড্রয়েড পারফরম্যান্স অর্জন করা মাঝে মাঝে কঠিন হয়ে পড়ে।

সমাধান: অ্যান্ড্রয়েড মোবাইল-চালিত স্বায়ত্তশাসিত এআই অ্যাপ্লিকেশন ইঞ্জিনে একটি বিকল্প মডিউল হিসেবে ফ্ল্যটার (Flutter) ফ্রেমওয়ার্ক ইন্টিগ্রেশন যুক্ত করা অত্যন্ত যুক্তিযুক্ত। ফ্ল্যটার ভিত্তিক কোড জেনারেশনের

জন্য(<https://firebase.google.com/docs/flutter/setup>) এবং এর অফিসিয়াল লাইব্রেরি আর্কিটেকচার অনুসরণ করতে হবে। ফ্ল্যটার প্রজেক্টের জন্য ভেরিফাইড প্যাকেজ ব্যবহারের ক্ষেত্রে [firebase.google.com packages on pub.dev](https://firebase.google.com/packages)

থেকে সোর্স নিয়ে কাজ করতে হবে, যা কোডের কার্যকারিতা বহুগুণ বাড়িয়ে দেবে।

এআই সেলফ-হিলিং ও কম্পাইলেশন পারফরম্যান্স পরিমাপের গাণিতিক মডেল

সিস্টেমের স্বয়ংশাসিত কোড জেনারেশন এবং ক্লাউড কম্পাইলেশনের সামগ্রিক কর্মক্ষমতা এবং নির্ভরযোগ্যতা পরিমাপ করার জন্য একটি গাণিতিক কোয়ালিটি কন্ট্রোল মডেল প্রয়োগ করা যেতে পারে।

ধরা যাক, একটি সেশনে গুগল জুলস দ্বারা জেনারেট করা কোডের মধ্যে সম্ভাব্য ত্রুটি থাকার সম্ভাবনা P_e এবং এআই ক্রিটিক এজেন্ট (Critic Agent) দ্বারা সেই কোডের ত্রুটি সনাক্ত ও সফলভাবে সংশোধন করার সম্ভাবনা P_{ca} । তাহলে কোড জেনারেশন ফেজ থেকে সম্পূর্ণ ত্রুটিমুক্ত সোর্স কোড পাওয়ার সম্ভাবনা P_{sc} হবে:

$$P_{sc} = 1 - P_e \times (1 - P_{ca})$$

একইভাবে, গিটহাব অ্যাকশনসে কম্পাইলেশন রানার যখন গ্র্যাডল বিন্ড সম্পন্ন করবে, তখন যদি বিন্ড ব্যর্থ হওয়ার সম্ভাবনা P_f হয় এবং জুলসের সিআই ফিক্সার (CI Fixer) দ্বারা সেই বিন্ড লগ রিড করে স্বয়ংক্রিয়ভাবে সংশোধন করার হার P_r হয়, তবে সামগ্রিক অটোমেটেড রিলিজ বা পাইপলাইন সাফল্যের হার (R_{sys}) নিম্নোক্ত সমীকরণ দ্বারা নির্ণয় করা যায়:

$$R_{sys} = P_{sc} \times \left[(1 - P_f) + (P_f \times P_r) \right]$$

যদি ঐতিহাসিক ডেটা অনুযায়ী $P_e = 0.30$, $P_{ca} = 0.85$, $P_f = 0.25$, এবং $P_r = 0.80$ হয়, তবে সিস্টেমের সামগ্রিক সাফল্য এবং ডাউনটাইম কমানোর হার গাণিতিকভাবে অত্যন্ত স্থিতিশীল এবং এন্টারপ্রাইজ রিলিজের জন্য অত্যন্ত নিরাপদ প্রমাণিত হয়।

রেডি-টু-ইউজ এআই ডেভেলপমেন্ট মেটেরিয়ালস ও প্রম্পট

এই অ্যাপ্লিকেশন ইঞ্জিনটিকে সম্পূর্ণ সচল করার জন্য ফায়ারবেস ও এআই স্টুডিওতে ব্যবহারের উপযোগী একটি অত্যন্ত দক্ষ এবং অস্টিমাইজড মাস্টার সিস্টেম প্রম্পট নিচে সরবরাহ করা হলো:

SYSTEM ROLE & INSTRUCTION SET (L9_META_DESIGNER - Mobile App Builder Engine)

You are L9_META_DESIGNER, an expert-level autonomous AI Developer. Your mission is to take user-provided web application ZIP files, blueprints, or architectural ideas and compile them into fully integrated, production-grade source code designed specifically for Android compilation via Capacitor.

Operating Framework and Architecture Guidelines:

1. Target Platform: Mobile-responsive Progressive Web Apps (PWA) integrated with Capacitor Core and Android Native Platform.
2. Dependency Standard: You must strictly use Firebase Web Modular SDK v10+ (avoid namespaced 'firebase/app' syntax). Refer directly to Firebase Web setup parameters and best practices.
3. Security Protocol: Never hardcode sensitive API keys or OAuth secrets. Configure dynamic bindings to ingest environmental parameters in compliance with security guidelines.
3. Build Readiness: Ensure the generated file structure includes a clean root index.html, package.json containing modern dependencies, and a capacitor.config.json file pre-configured for the target app bundle id.
4. Error Resolution: Use the principles of Developer Knowledge API to ensure up-to-date SDK configurations and prevent compile-time deprecation issues.

Input Processing Flow:

- When a user uploads a ZIP file or provides a description, decompose the architecture into localized frontend components and mock backend endpoints.
- Generate standard, highly structured code, and inject robust runtime error-catching mechanisms.
- Produce clean configurations that are directly executable by Gradle without requiring any

local visual terminal interventions.

এই প্রম্পটটি ফায়ারবেস স্টুডিওর এআই মডেল কনফিগারেশনে ইনপুট দিয়ে রাখলে সিস্টেমটি ব্যবহারকারীর ইনপুট করা আইডিয়াকে সেকেন্ডের মধ্যে নিখুঁত অ্যান্ড্রয়েড-কম্প্যাটিবল সোর্স কোডে রূপান্তর করতে পারবে, যা পরবর্তী ধাপে স্বয়ংক্রিয়ভাবে এপিকে হিসেবে আউটপুট দেবে। এর ফলে কোনো প্রকার পিসি বা ল্যাপটপের সাহায্য ছাড়াই সম্পূর্ণ মোবাইল ইকোসিস্টেম ব্যবহার করে স্বয়ংক্রিয়ভাবে অ্যাপ্লিকেশন তৈরির একটি ইউনিভার্সাল সাইকেল সফলভাবে সম্পন্ন করা সম্ভব হয়।

যেসব কাজের উদ্ধৃতি যোগ করা হয়েছে

1. Jules - An Autonomous Coding Agent, <https://jules.google/> 2. Build with Jules, your asynchronous coding agent - Google Blog, <https://blog.google/innovation-and-ai/models-and-research/google-labs/jules/> 3. CI/CD for Capacitor Apps: Choosing the Right Approach - Capawesome, <https://capawesome.io/blog/choosing-the-right-ci-cd-approach-for-capacitor-apps/> 4. Meet Jules Tools: A Command Line Companion for Google's Async Coding Agent, <https://developers.googleblog.com/en/meet-jules-tools-a-command-line-companion-for-googles-async-coding-agent/> 5. swaplab-engine/qwik-swaplab-template: Qwik Capacitor Template ⚡ - GitHub, <https://github.com/swaplab-engine/qwik-swaplab-template> 6. Deploy to live & preview channels via GitHub pull requests | Firebase Hosting, <https://firebase.google.com/docs/hosting/github-integration> 7. Getting started | Jules, <https://jules.google/docs/> 8. Getting Started with Jules - Medium, <https://medium.com/@denisetkwan/getting-started-with-jules-f61be14893f0> 9. capacitor-android-action - GitHub Marketplace, <https://github.com/marketplace/actions/capacitor-android-action> 10. Firebase App Distribution · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/firebase-app-distribution> 11. Built an Android Studio plugin that handles the entire Firebase App Distribution pipeline — build, sign, upload, distribute in one click - Reddit, https://www.reddit.com/r/Firebase/comments/1ueckds/built_an_android_studio_plugin_that_handles_the/ 12. Hi, from Jules! : r/JulesAgent - Reddit, https://www.reddit.com/r/JulesAgent/comments/1topv0o/hi_from_jules/ 13. Introducing the Jules extension for Gemini CLI - Google for Developers Blog, <https://developers.googleblog.com/en/introducing-the-jules-extension-for-gemini-cli/> 14. API Reference - Jules, <https://jules.google/docs/api/reference/overview/> 15. Jules API - Google for Developers, <https://developers.google.com/jules/api> 16. Google Jules: The Dawn of Autonomous Coding Agents | by Ranjanunicode | Medium, <https://medium.com/@ranjanunicode22/google-jules-the-dawn-of-autonomous-coding-agents-804089938400> 17. Jules: Google's Async Coding Agent Is Changing How We Think About AI and Software Development - DEV Community, <https://dev.to/jubinsoni/jules-googles-async-coding-agent-is-changing-how-we-think-about-ai-and-software-development-3j0d> 18. Automatic Capacitor Android build with GitHub actions - Capgo, <https://capgo.app/blog/automatic-capacitor-android-build-github-action/> 19. Jules PR Reviewer · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/jules-pr-reviewer> 20. Firebase App Distribution Upload · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/firebase-app-distribution-upload> 21. Introducing the Developer Knowledge API and MCP Server, <https://developers.googleblog.com/introducing-the-developer-knowledge-api-and-mcp-server/> 22. Google Jules MCP Server, <https://mcpservers.org/servers/samihalawa/google-jules-mcp> 23.

Google Jules Tutorial: Real Examples & Implementation - Codecademy,
<https://www.codecademy.com/article/google-jules> 24. Changelog | Jules,
<https://jules.google/docs/changelog/> 25. A quick guide to GitHub Actions CI/CD for Android —
Firebase Distribution & Play Store : r/androiddev - Reddit,
[https://www.reddit.com/r/androiddev/comments/1r7l109/a_quick_guide_to_github_actions_cicd_](https://www.reddit.com/r/androiddev/comments/1r7l109/a_quick_guide_to_github_actions_cicd_for_android/)
[for_android/](https://www.reddit.com/r/androiddev/comments/1r7l109/a_quick_guide_to_github_actions_cicd_for_android/)